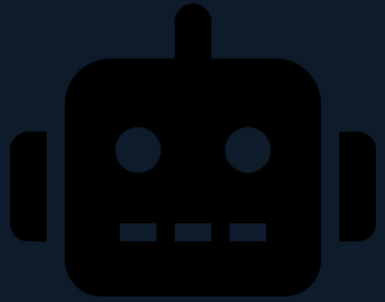
SESSION 12

AIOps Masterclass

Automated Remediation

& Self-Healing Systems



- Remediation Patterns
- Closed-Loop Automation
- Self-Healing Architectures

Session Agenda

01

Remediation Patterns

Reactive, proactive & prescriptive patterns

02

Pattern Deep-Dives

Runbook, auto-scale, circuit breaker & more

03

Closed-Loop Automation

Detect → Diagnose → Decide → Act → Verify

04

Closed-Loop Implementation

Engines, orchestrators & guardrails

05

Self-Healing Architectures

Principles, pillars & design patterns

06

Self-Healing in Practice

Kubernetes, chaos engineering & observability

What is Automated Remediation?

Automated remediation is the use of AI-driven systems to detect, diagnose, and resolve IT incidents without—or with minimal—human intervention, reducing MTTR and improving system reliability.

Detect

Real-time anomaly & fault detection via ML-powered observability

Diagnose

Root cause analysis linking symptoms to underlying failures

Resolve

Automated execution of corrective actions via runbooks or agents

PART

01
Remediation Patterns

Understanding reactive, proactive, and prescriptive remediation strategies in AIOps

- → Pattern taxonomy & classification
- → Common patterns with use-cases
- → Pattern selection framework

Remediation Pattern Taxonomy

Reactive

- Incident runbook execution
- Auto-restart failing services
- Rollback on deployment failure
- Alert storm suppression

Proactive

- Capacity right-sizing
- Predictive scaling
- Certificate renewal automation
- Disk cleanup scheduling

Prescriptive

- ML-recommended remediation
- Context-aware action selection
- Risk-scored auto-approval
- Multi-step orchestration

← *Trigger Time Horizon* →

Core Remediation Patterns

Runbook Automation

Codifies expert knowledge into repeatable scripts triggered by alerts

Auto-Scaling

Dynamically adjusts compute resources based on load predictions

Circuit Breaker

Prevents cascade failures by halting calls to degraded dependencies

Blue/Green Rollback

Instant traffic switch to stable version on anomaly detection

Self-Restart / Watchdog

Automatically restarts crashed processes or unhealthy containers

Data Pipeline Retry

Idempotent re-execution with exponential backoff on transient failures

Pattern Selection Framework

Factor	Reactive	Proactive	Prescriptive
Trigger	Alert / event	Scheduled / trend	ML recommendation
Data needed	Current metrics	Historical + live	Full observability
Latency	Seconds	Minutes to hours	Real-time to minutes
Risk	Low (scripted)	Medium	High (AI decides)
Best for	P1 incidents	Capacity & hygiene	Complex root-cause

Real-World Scenario: Memory Leak Remediation

Detect

Memory > 90% for 5 min
Anomaly score: 0.94

Correlate

Links to pod
user-service-7c9f
Deployment v2.1.3

Decide

Confidence: 87%
Action: Rolling restart
Approval: Auto

Execute

kubectl rollout
restart
Duration: 23s

Verify

Memory: 42%
SLA restored
Incident closed

⚡ **MTTR reduced from ~45 minutes (manual) → 23 seconds (automated) | SLA breach averted**

PART

02

Closed-Loop Automation

Building feedback-driven control loops that continuously detect, decide, act, and learn

- → Detect-Diagnose-Decide-Act-Verify cycle
- → Automation engines & orchestration
- → Guardrails, approval gates & rollback

The Closed-Loop Control Cycle

1. Detect

Metrics, logs, traces
anomaly detection

2. Diagnose

Root cause analysis
correlation engine

3. Decide

Policy engine
ML action scoring

4. Act

Runbook execution
API orchestration

5. Verify

Outcome validation
feedback to model

Guardrails, Gates & Safe Automation

Why Guardrails Matter

- Autonomous systems can amplify mistakes at machine speed
- Blast radius control: limit scope of auto-actions
- Regulatory compliance requires audit trails
- Trust must be earned before full autonomy

Gate Types

Confidence Threshold

Act only if ML model confidence > configurable %

Change Freeze Window

Block automation during business-critical periods

Blast Radius Limit

Max N pods / services affected in one action

Human-in-the-Loop

Approval required for Severity-1 or irreversible acts

Closed-Loop Tooling Landscape

Observability	Orchestration	Cloud-Native	Policy & Governance
Dynatrace Davis AI	Ansible AWX / AAP	Kubernetes HPA/VPA	Open Policy Agent
Datadog Watchdog	Rundeck (PagerDuty)	AWS Systems Manager	HashiCorp Sentinel
New Relic AIOps	StackStorm	GCP Operations	Chef InSpec
Elastic Observability	ServiceNow Flow	Azure Automation	AWS Config Rules

Selection criteria: integration depth, ML capability, human-override controls, audit logging, and ITSM integration

PART

03

Self-Healing Architectures

Designing systems that autonomously detect, adapt, and recover without human intervention

- → Self-healing principles & properties
- → Architecture patterns & blueprints
- → Kubernetes & cloud-native self-healing

Properties of Self-Healing Systems

Self-Configuring

Automatically adapts to environmental changes and new workloads without manual tuning

Self-Optimizing

Continuously tunes performance parameters toward defined objectives (cost, latency, SLA)

Self-Protecting

Detects and defends against malicious attacks and cascade failures proactively

Self-Healing

Discovers and recovers from faults automatically, restoring desired state

Self-Healing Architecture Blueprint

Observation Layer

Metrics · Logs · Traces · Events · Business KPIs

Intelligence Layer

Anomaly Detection · RCA Engine · Decision Model · Policy Store

Action Layer

Runbook Engine · K8s Operator · IaC Actuator · ITSM Integration

Key principle: Each layer feeds signals upward and actions downward. The feedback loop between Action → Observation is what makes the architecture truly self-healing rather than just automated.

Kubernetes: Self-Healing in Practice

Liveness Probes

Restart container when application is stuck/unresponsive

```
livenessProbe: httpGet / exec / tcpSocket
```

Readiness Probes

Remove unhealthy pods from service endpoints without restart

```
readinessProbe: controls traffic routing
```

ReplicaSet Controller

Continuously reconciles desired vs actual pod count

```
kubectl get rs → auto-recreates deleted pods
```

HPA – Horizontal Pod Autoscaler

Scale pods on CPU/memory/custom metrics automatically

```
target: averageUtilization: 70
```

Node Problem Detector

Taints unhealthy nodes, triggers pod rescheduling

```
node-problem-detector DaemonSet
```

Observability as the Nervous System

Metrics

What is happening?

- Prometheus / OTEL
- RED method (Rate, Errors, Duration)
- USE method (Utilization, Saturation, Errors)
- Custom business KPIs

Logs

Why did it happen?

- Structured JSON logging
- Log anomaly detection (ML)
- Correlation with trace IDs
- Fluentd / Loki / Splunk

Traces

Where did it happen?

- Distributed tracing (Jaeger/Tempo)
- Service dependency maps
- Latency breakdown per service
- Error propagation path

Self-Healing Design Patterns

01 Sidecar Health Agent

Co-located container monitors & restarts main service on failure

02 Bulkhead Pattern

Isolate service pools to prevent single failure from cascading

03 Retry with Backoff

Automatic retry with exponential delay for transient faults

04 Health Endpoint Monitor

External probe verifies /health endpoints, routes around failures

05 Leader Election

Automatic failover to standby leader in distributed systems

06 Saga Pattern

Compensating transactions auto-rollback failed distributed workflows

AIOps Automation Maturity Model

← *Most organizations today* → ← *Target state* →

Manual	Assisted	Semi-Automated	Automated	Autonomous
All remediation by humans; alerts trigger ticket creation	Runbooks suggested; engineers choose & execute manually	Low-risk runbooks auto-execute; human approval for critical	Most incidents auto-remediated; humans notified post-action	AI-driven self-healing; humans set policy only

Session 12 — Key Takeaways

Remediation Patterns

Choose reactive, proactive, or prescriptive patterns based on trigger horizon, data availability, and risk tolerance

Closed-Loop Automation

MAPE-K and Detect→Diagnose→Decide→Act→Verify cycles are the foundation; guardrails prevent runaway automation

Self-Healing Architecture

Layer observation, intelligence, and action; feedback between layers is what makes a system truly self-healing

Start with Trust, then Autonomy

Begin at L2–L3 maturity; earn confidence through audit trails, approval gates, and measurable MTTR improvement